



NLP: Large Language Models (LLM)



NLP

- Machine translation.
- Information retrieval (e.g., search engines).
- Sentiment analysis (e.g., positive, negative, happiness, sadness, etc.).
- Information extraction (e.g., summary, keywords, etc.).
- Text generation.



Concept: Tokens

- Individual pieces of text such as:
 - Words.
 - Punctuation.
 - Names (ChatGPT, Transformer...).
 - Abbreviations (LSTM, NATO, EU...).
- For example:
 - “This LLM is amazing!” is tokenized to [“This”, “LLM”, “is”, “amazing”, “!”].



The success of ChatGPT

1. Massive amounts of data.
2. Significant computing power.
3. Word embeddings.
4. Self-attention.
5. Transformers.
6. Reinforcement Learning from Human Feedback (RLHF).



The success of ChatGPT

1. Massive amounts of data.
2. Significant computing power.
3. Word embeddings. —————→ 1980; 2013 (Word2vec); 2014 (GloVe)
4. Self-attention. —————→ 2014; 2017 (Transformers)
5. Transformers. —————→ 2017
6. Reinforcement Learning from Human Feedback (RLHF).



The success of ChatGPT

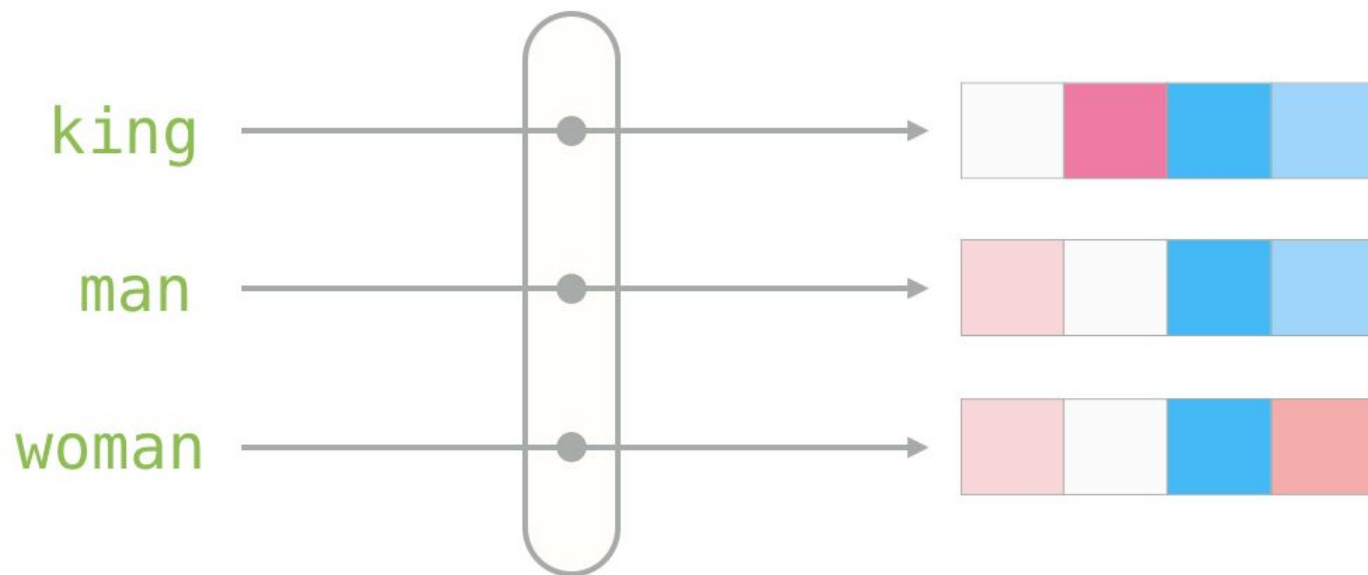
1. Massive amounts of data.
2. Significant computing power.
3. Word embeddings.
4. Self-attention.
5. Transformers.
6. Reinforcement Learning from Human Feedback (RLHF).



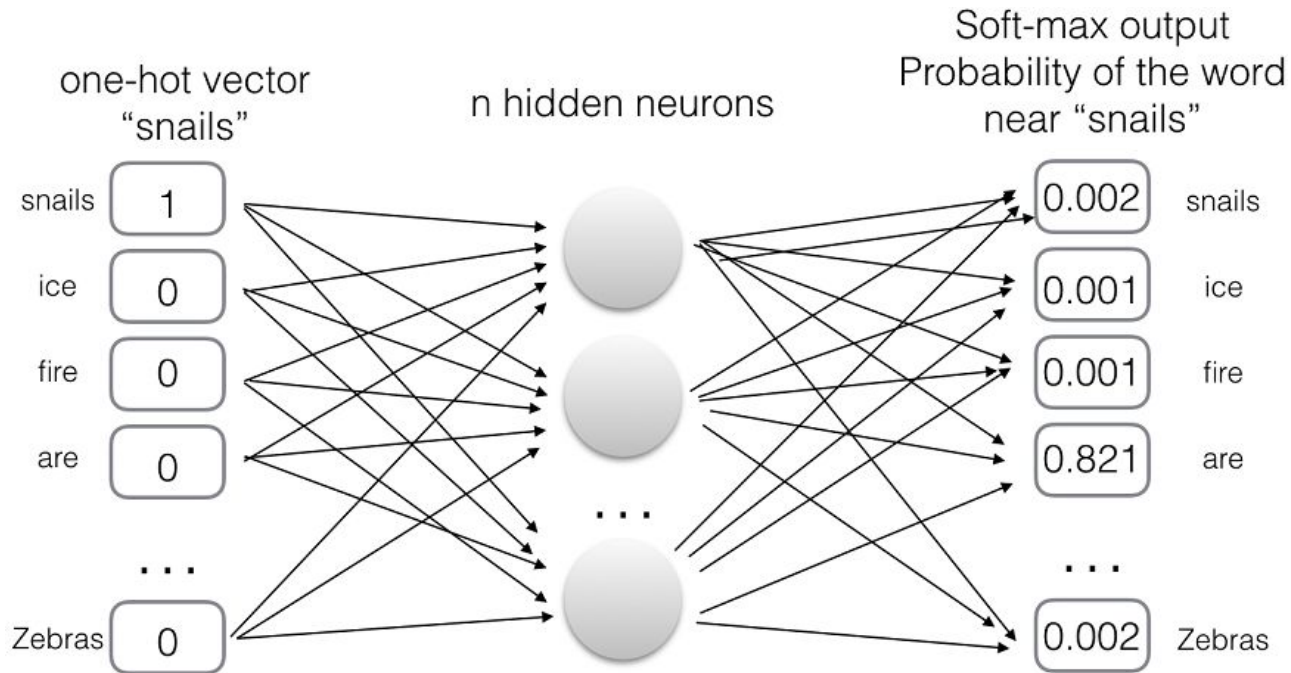
Word embedding

- Representation of words, typically in N-dimensional vectors.
 - Words that share common contexts in the corpus have similar vectors.
 - More dissimilar words are located further
- Word2vec and GloVe are different algorithms for computing word embeddings.

Word2vec



Word2vec





Word2vec

Model architectures:

- Continuous Bag-Of-Words (CBOW).
- Skip-gram.



Word2vec – CBOW

1. Identify all V unique words in the corpus.



Word2vec – CBOW

1. Identify all V unique words in the corpus.
2. Choose the dimensionality N for representing the word embeddings.



Word2vec – CBOW

1. Identify all V unique words in the corpus.
2. Choose the dimensionality N for representing the word embeddings.
3. Neural Network architecture:
 - a. Input layer of size V .
 - b. Hidden layer of size N .
 - c. Output layer of size V .



Word2vec – CBOW

1. Identify all V unique words in the corpus.
2. Choose the dimensionality N for representing the word embeddings.
3. Neural Network architecture:
 - a. Input layer of size V .
 - b. Hidden layer of size N .
 - c. Output layer of size V .
4. Task:
 - a. Input: vector of size V , representing the context of a word.
 - b. Output: Predict the desired word.



Word2vec – CBOW

- Assuming 2 words as context, the input to the NN is:
 - Compute the one-hot encoded vector of length V for each context word.
 - Compute the average of such vectors.



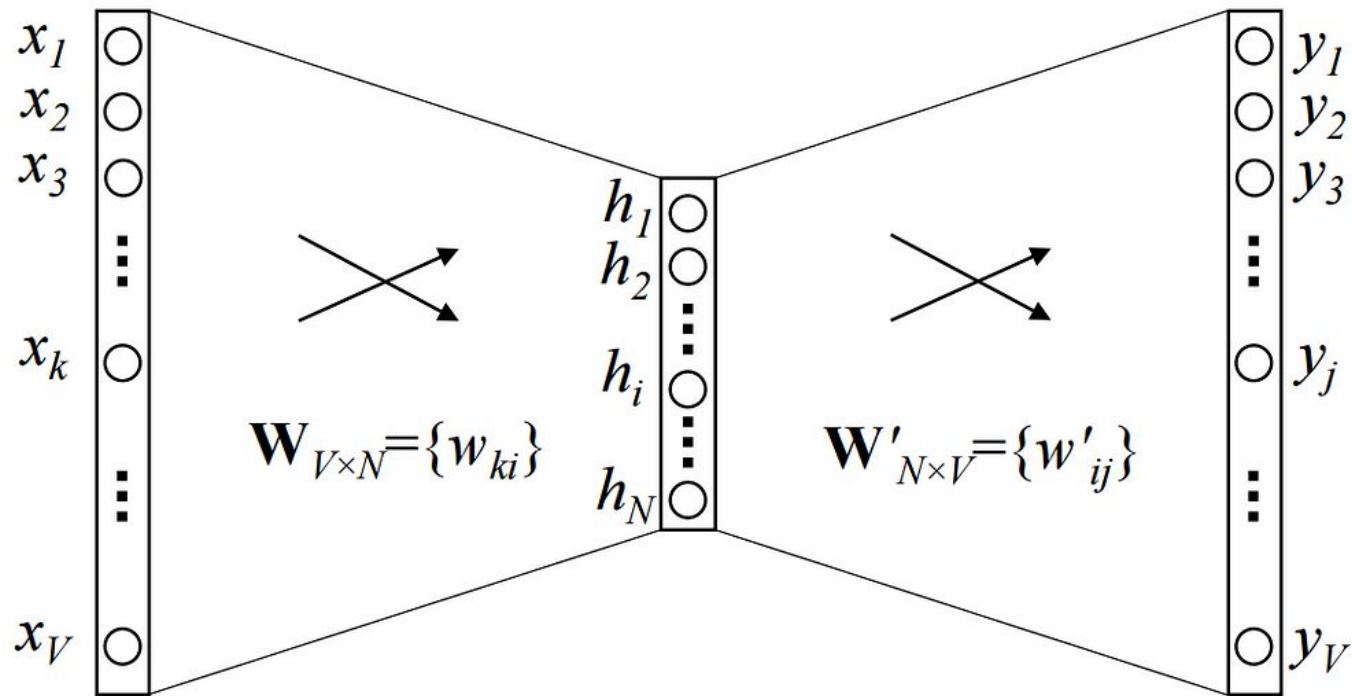
Word2vec – CBOW

- Example: “the fierce gray wolf hunts”
 - Target: “gray”
 - Context of 2:
 - “fierce” $\rightarrow [0, 1, 0, 0, 0]$
 - “wolf” $\rightarrow [0, 0, 0, 1, 0]$
 - NN input = average of vectors “fierce” and “wolf”:
 - input $\rightarrow [0, \frac{1}{2}, 0, \frac{1}{2}, 0]$

Input layer

Hidden layer

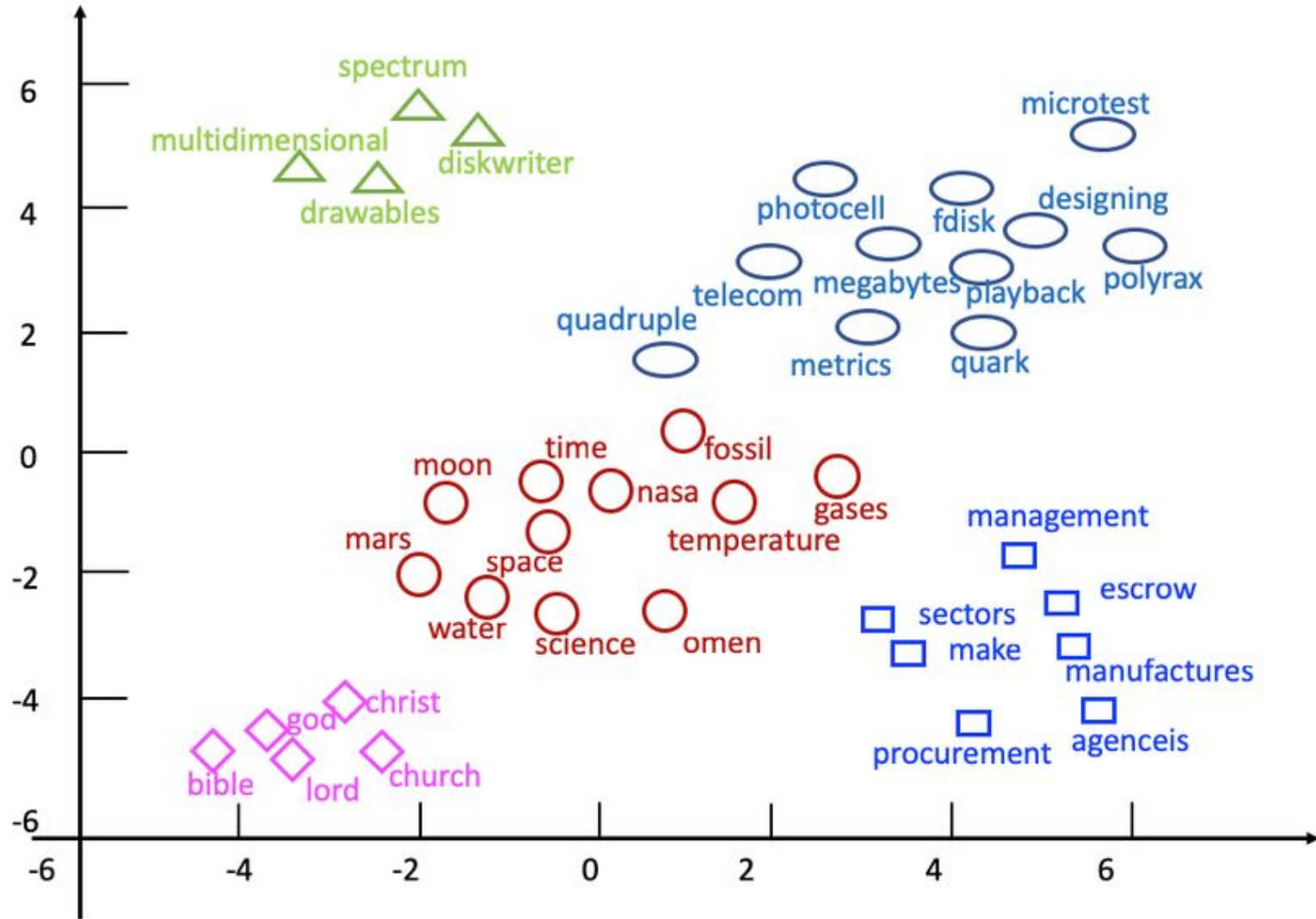
Output layer

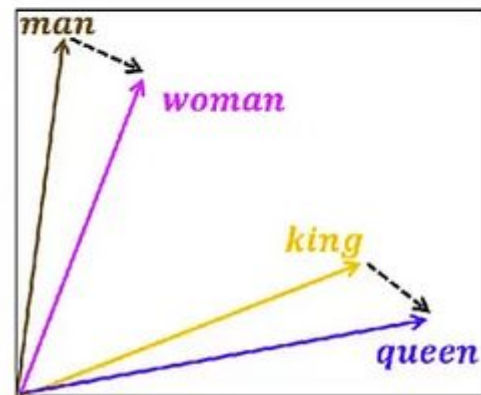
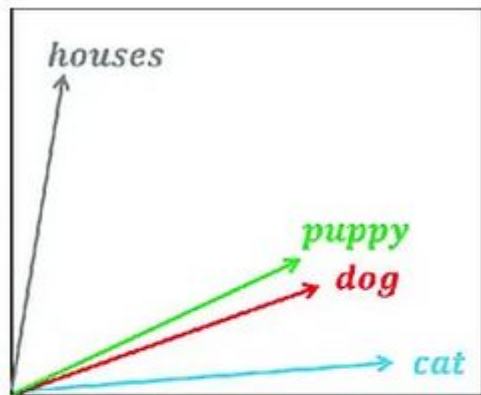


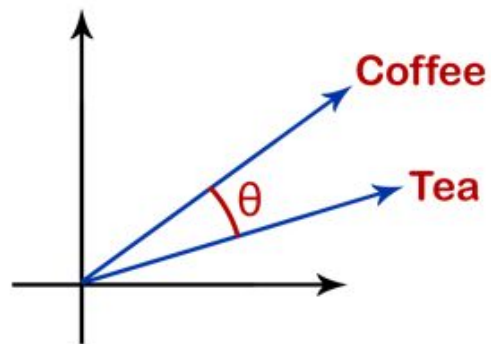
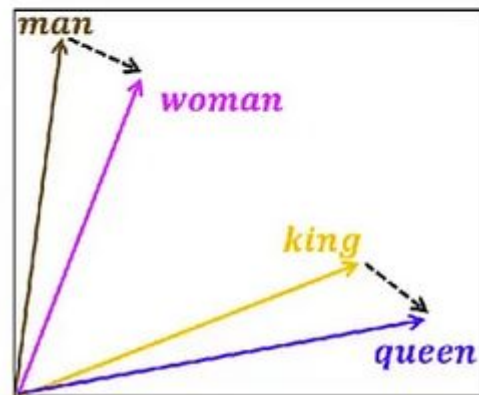
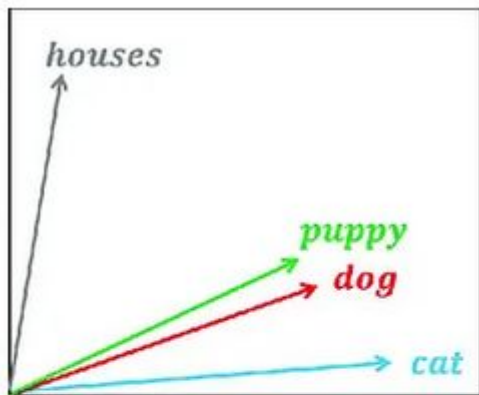


Word2vec – CBOW

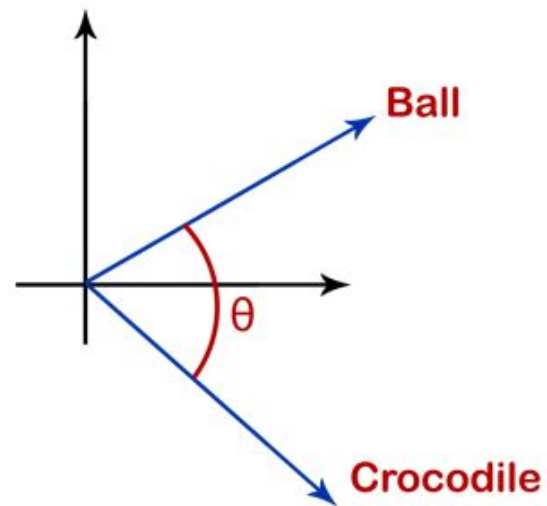
- After training with a lot of text, the NN has learned the weights of abstract representations of words and context.
- For each word in the corpus, the embedded vector representation of length N is:
 - The weights associated to that word, from the input to the hidden layer.
 - E.g.,
 - The one-hot encoded vector of “wolf” is $[0, 0, 0, 1, 0]$.
 - The embedded vector for “wolf” is the N weights associated with the 4th input neuron.
 - Context words that lead to the same target should have similar weights.







$$\text{sim}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$





Resources

- <https://www.youtube.com/watch?v=UqRCEmrv1gQ>



The success of ChatGPT

1. Massive amounts of data.
2. Significant computing power.
3. Word embeddings.
4. Self-attention.
5. Transformers.
6. Reinforcement Learning from Human Feedback (RLHF).



Self-attention

- Applied in a sequence of words.
- Allows to pay attention to similar/important words in the sequence.
- In contrast, RNNs process words sequentially, favoring more recent words.

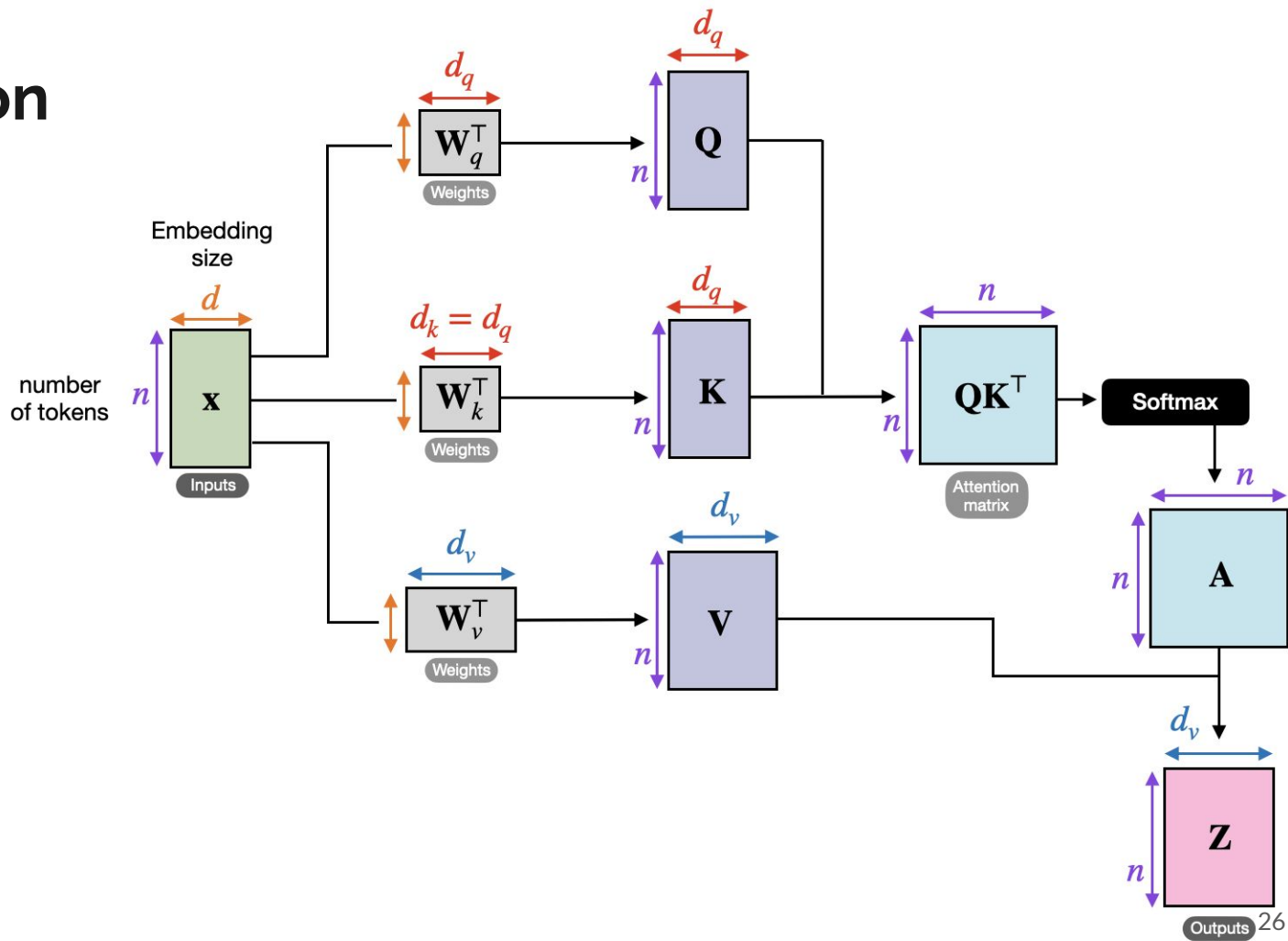


Self-attention

- Applied in a sequence of words.
- Allows to pay attention to similar/important words in the sequence.
- In contrast, RNNs process words sequentially, favoring more recent words.

The young boy always carries his toy car with ? → him

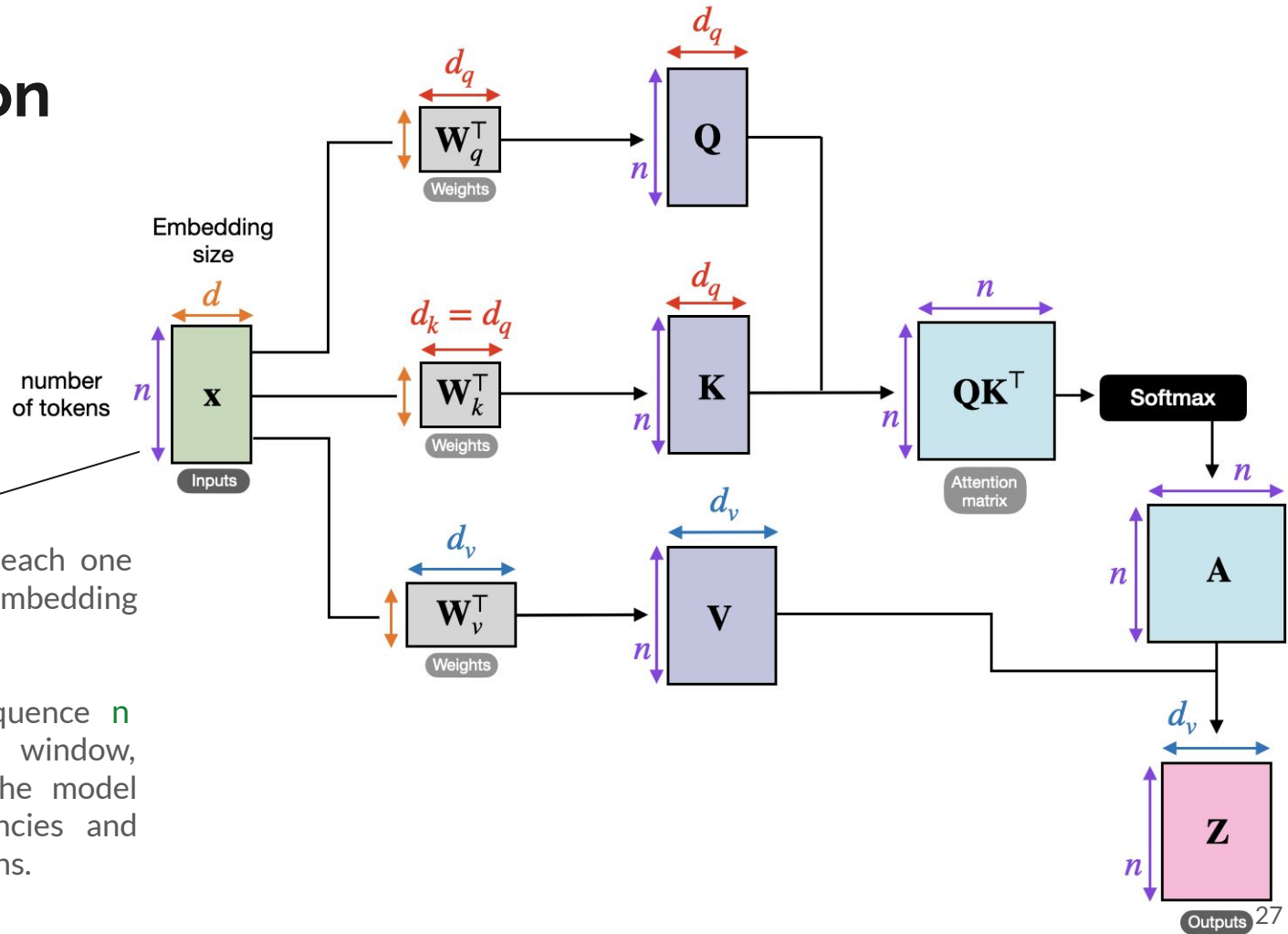
Self-Attention



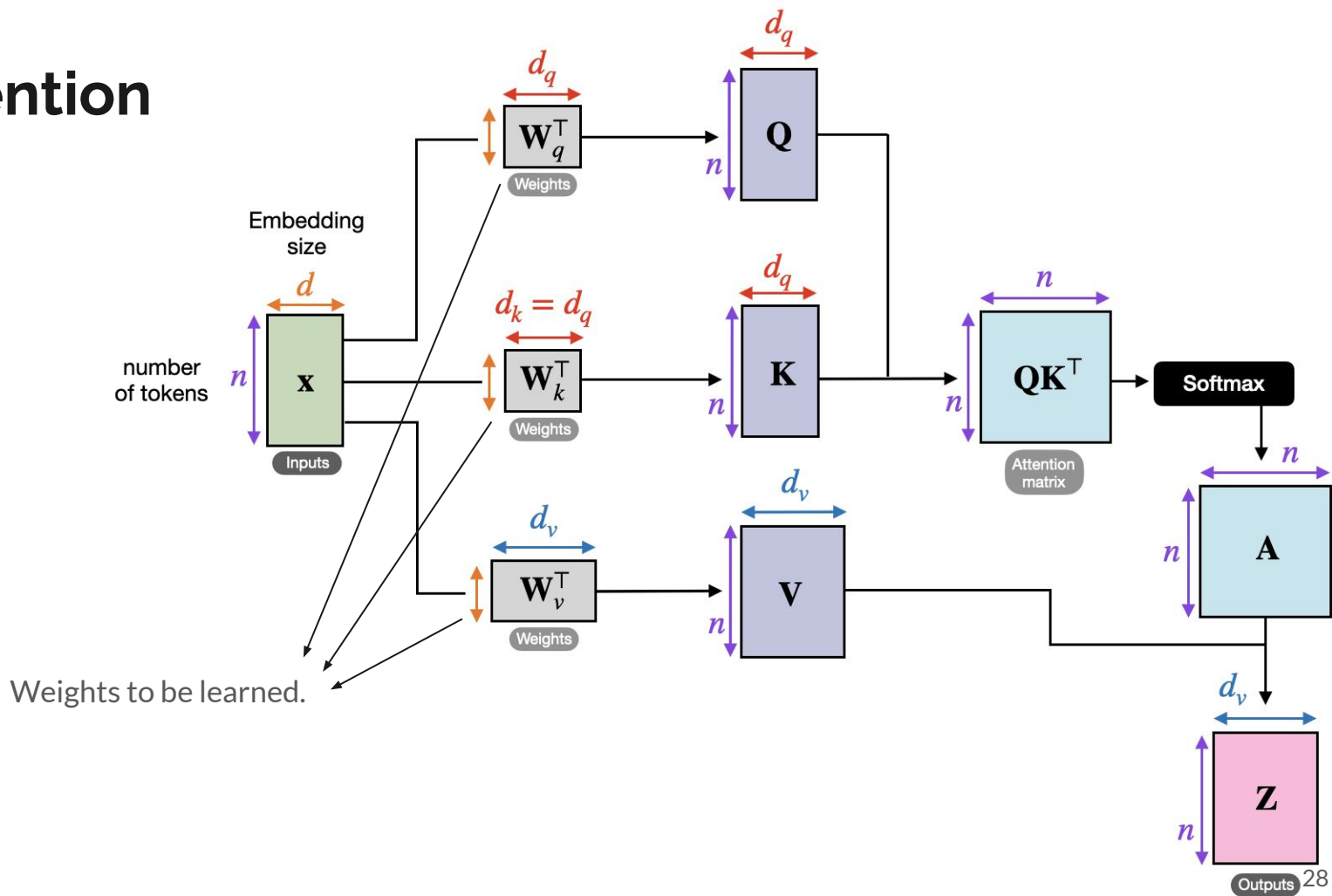
Self-Attention

Sequence of n tokens, each one represented with an embedding vector of d numbers.

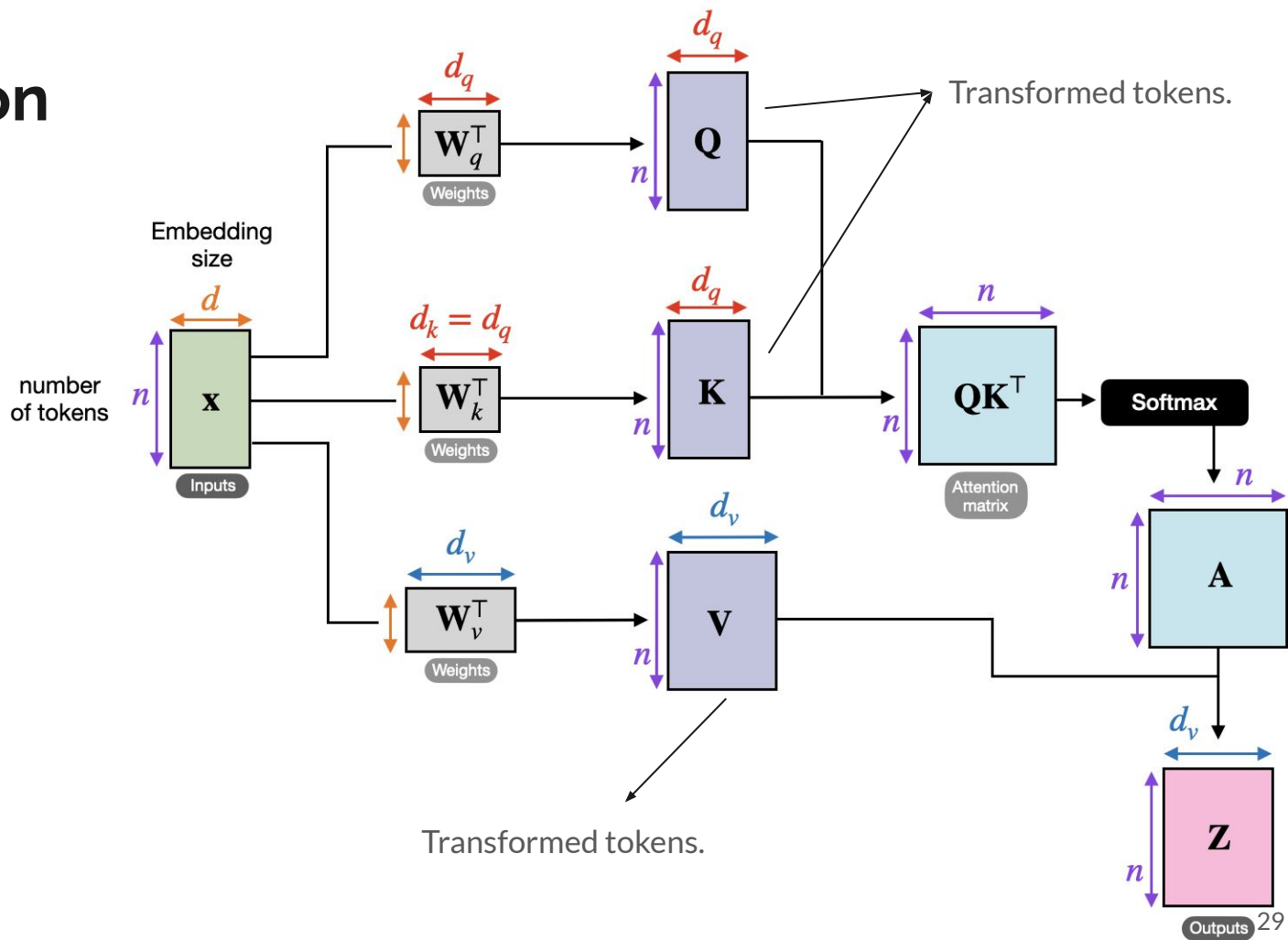
The length of the sequence n defines the context window, determining how far the model can consider dependencies and patterns across the tokens.



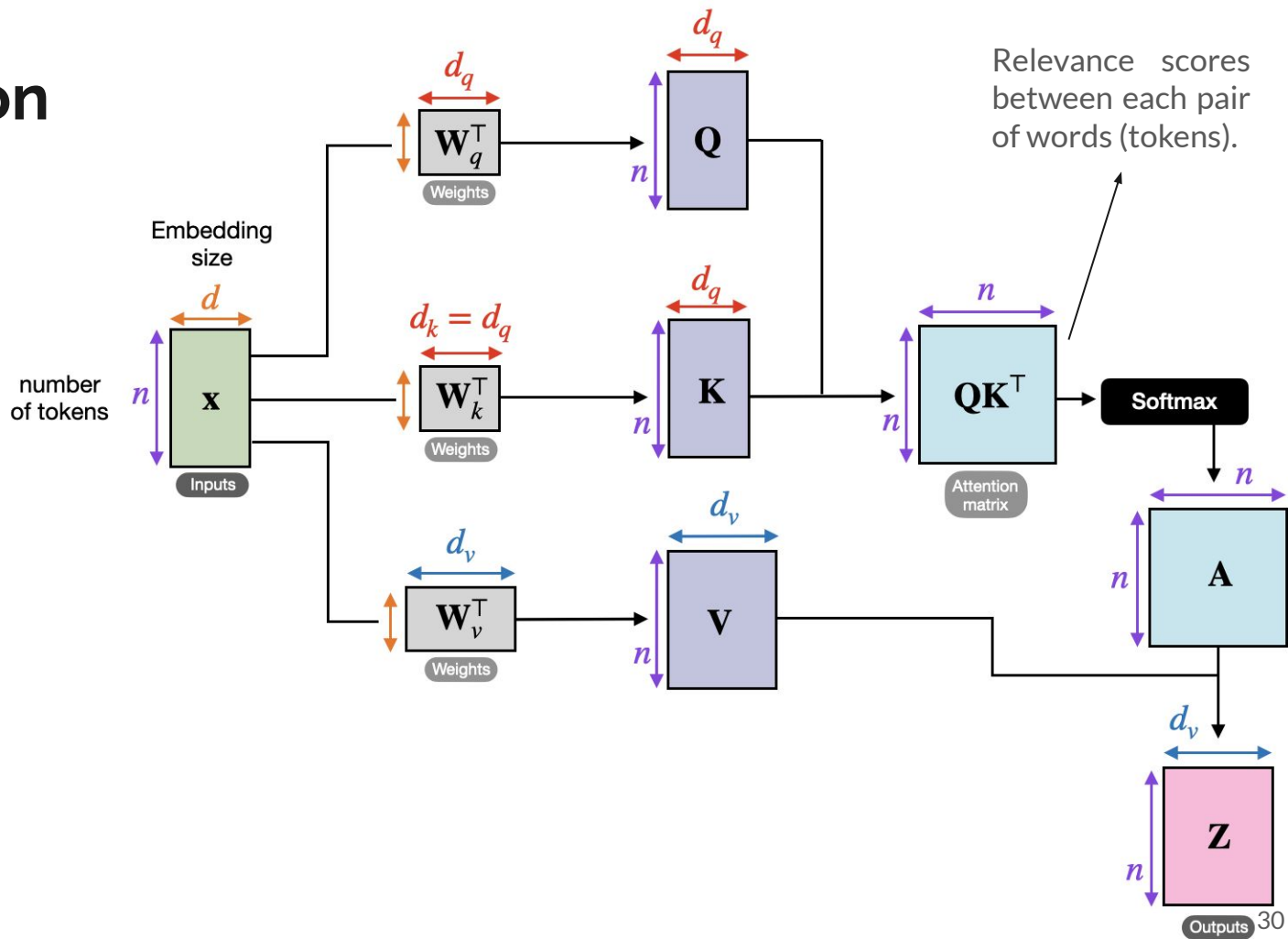
Self-Attention



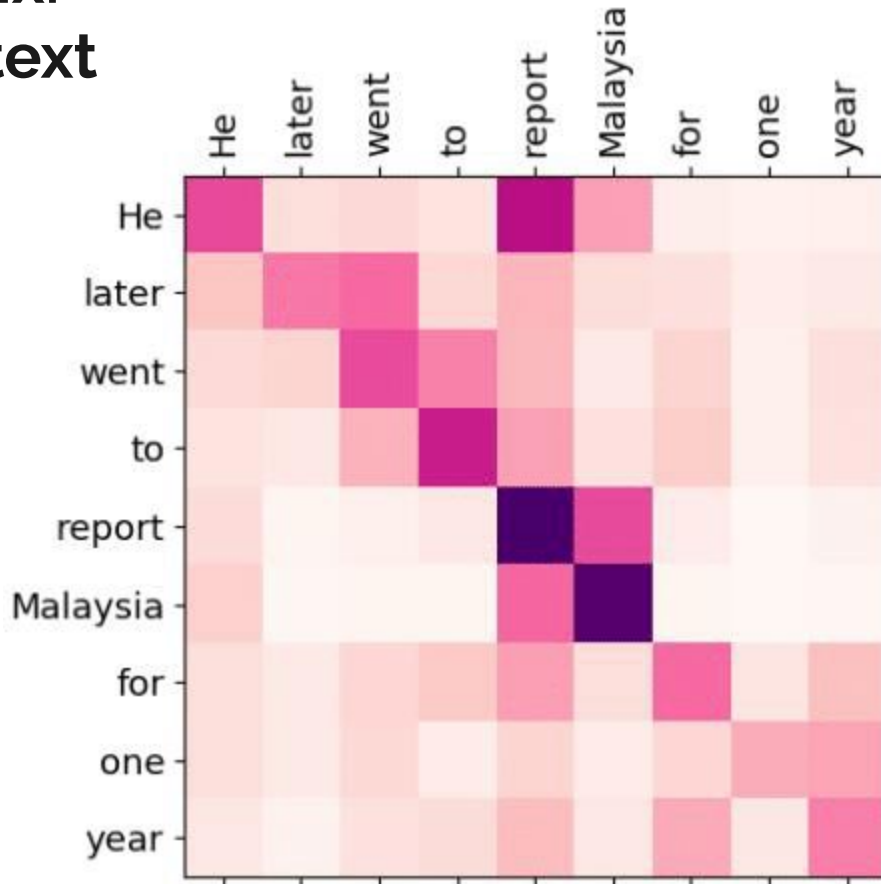
Self-Attention



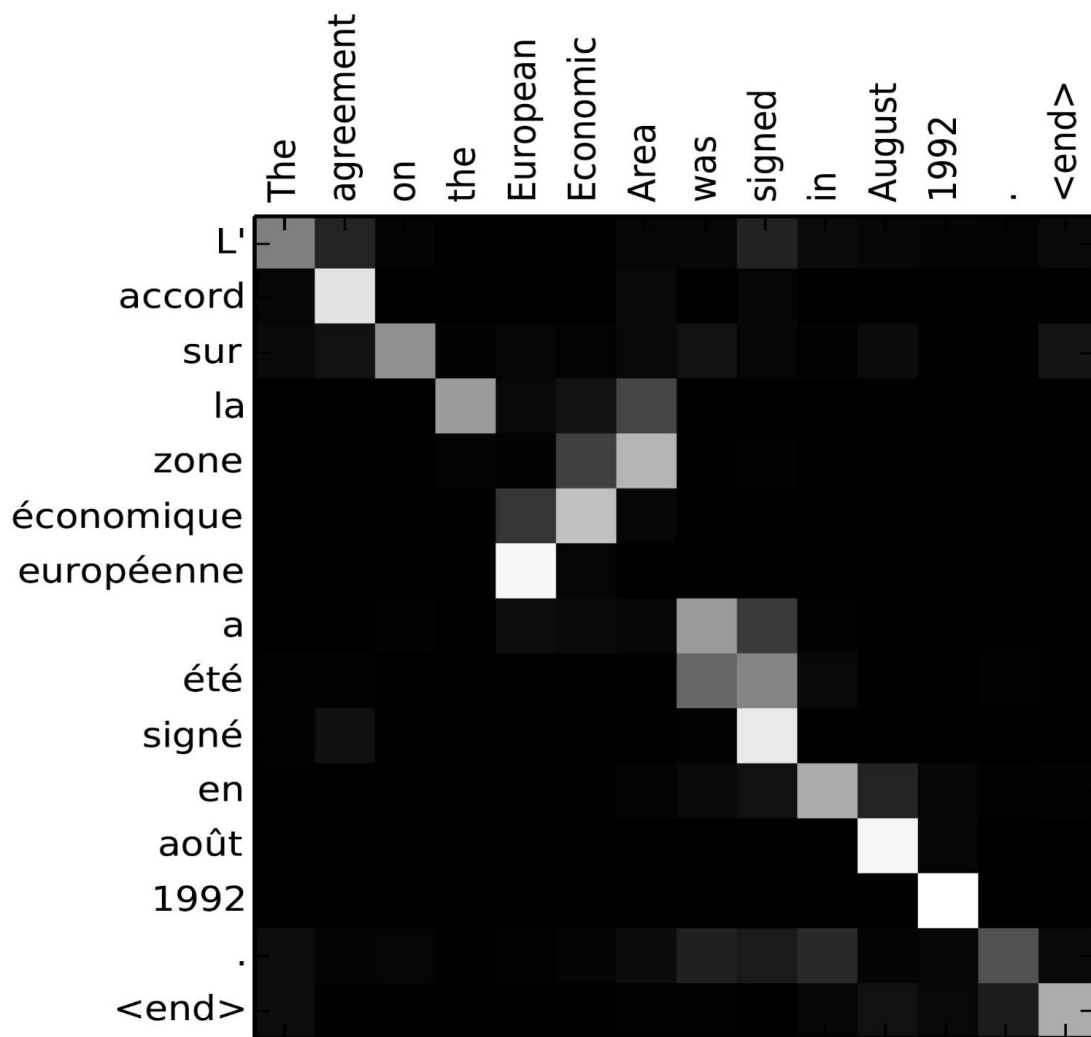
Self-Attention



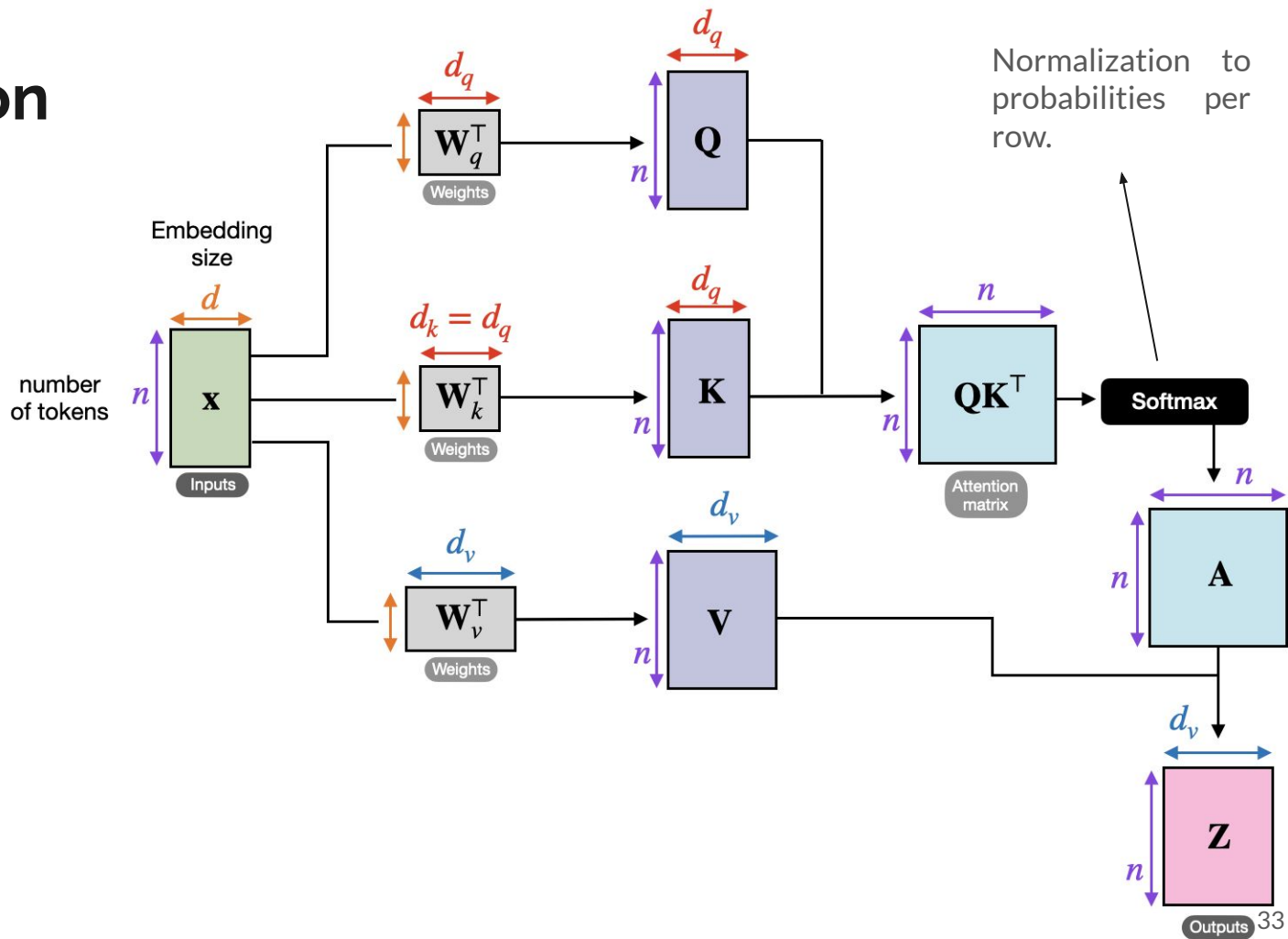
Attention Matrix: Semantic Context



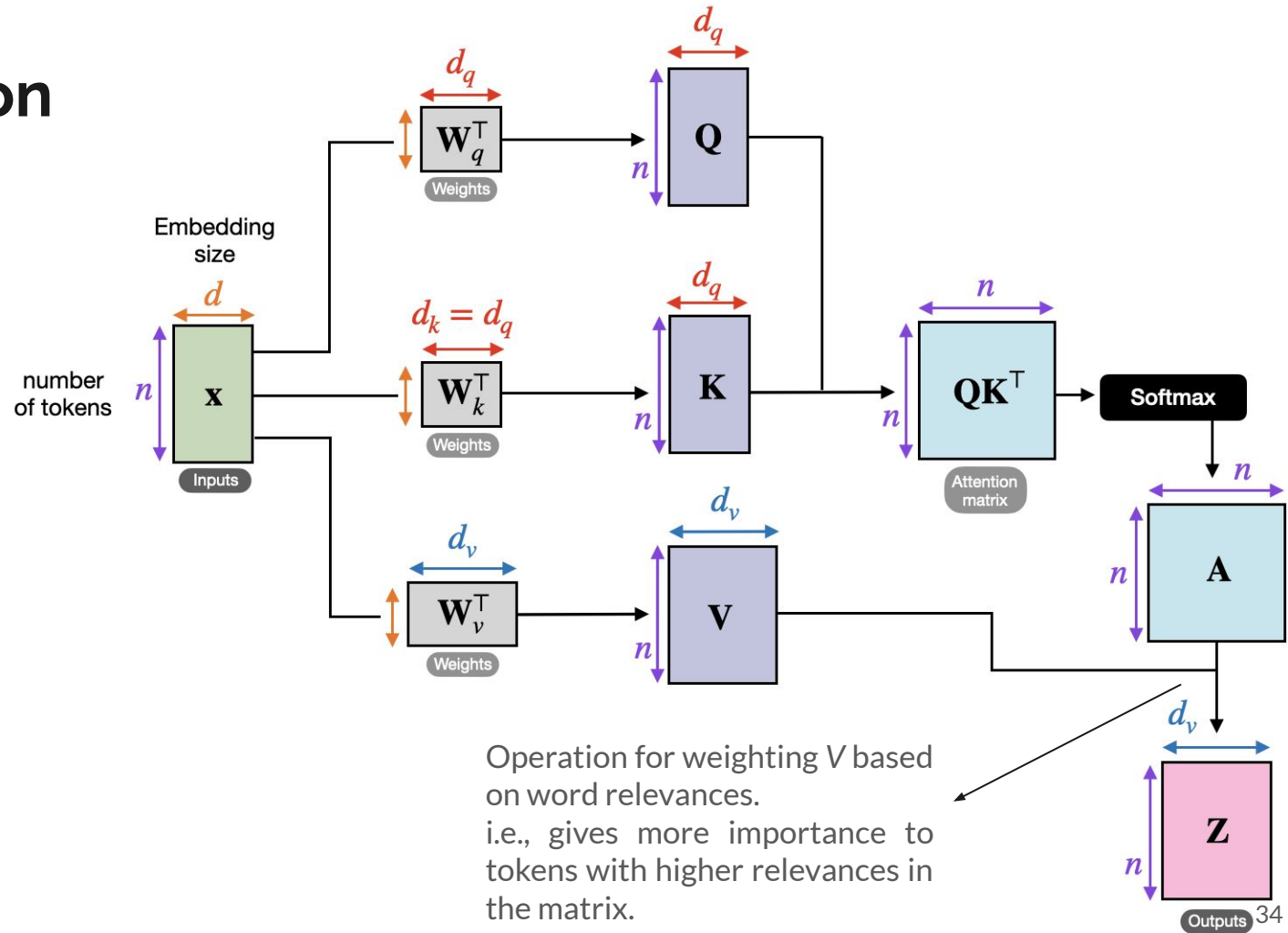
Attention Matrix: Translation



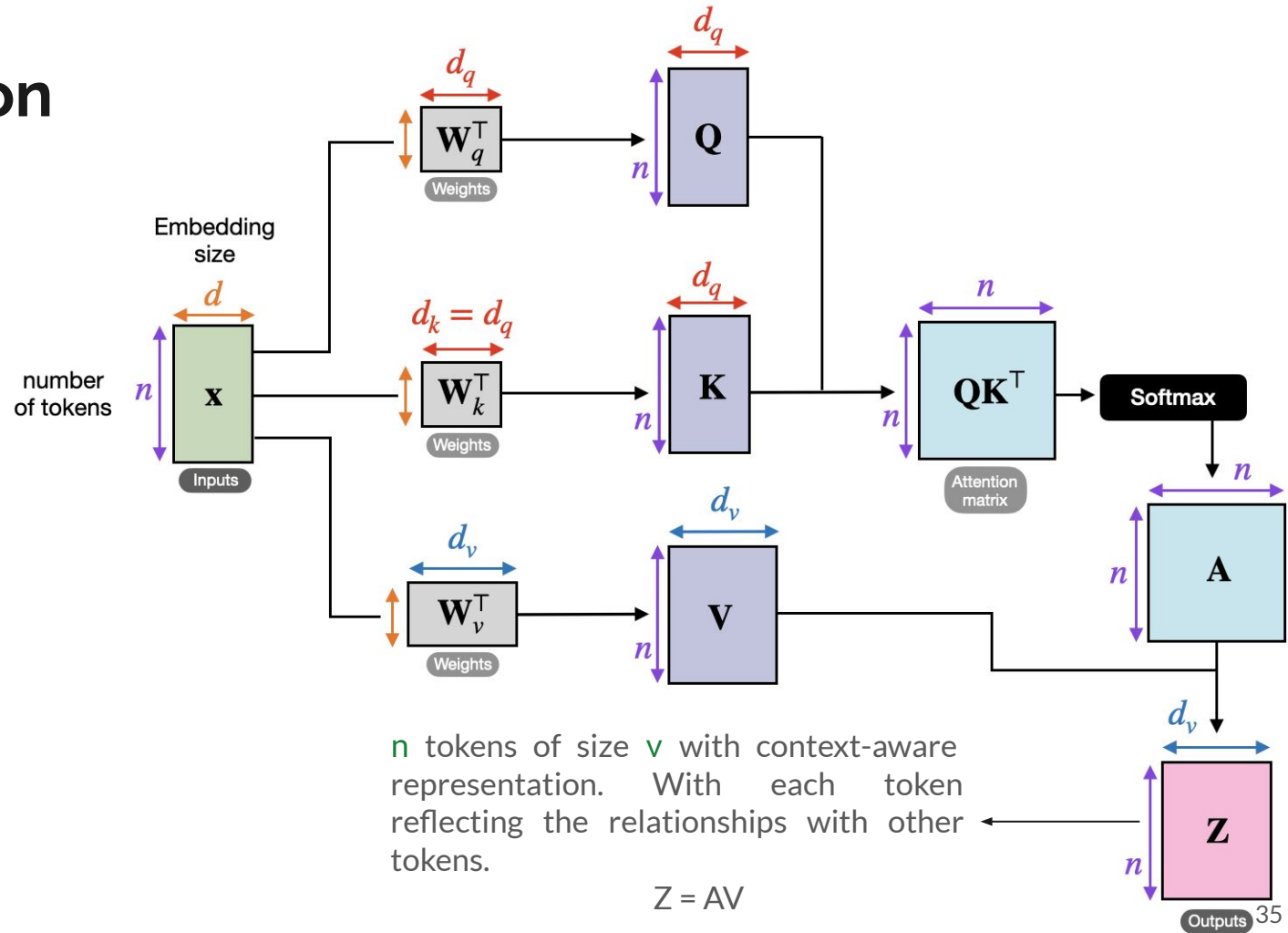
Self-Attention



Self-Attention



Self-Attention





Self-attention

- Pros:
 - Contextual understanding.
 - Operations between tokens can be applied in parallel.
- Cons:
 - Computationally and memory intensive for long sequences.
 - Does not consider sequential (positional) importance.



Resources

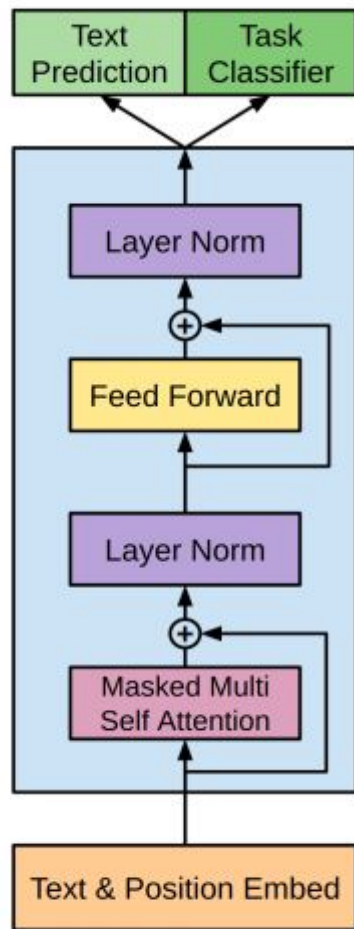
- <https://www.linkedin.com/pulse/gpt-4-explaining-self-attention-mechanism-fatos-ismali/>



The success of ChatGPT

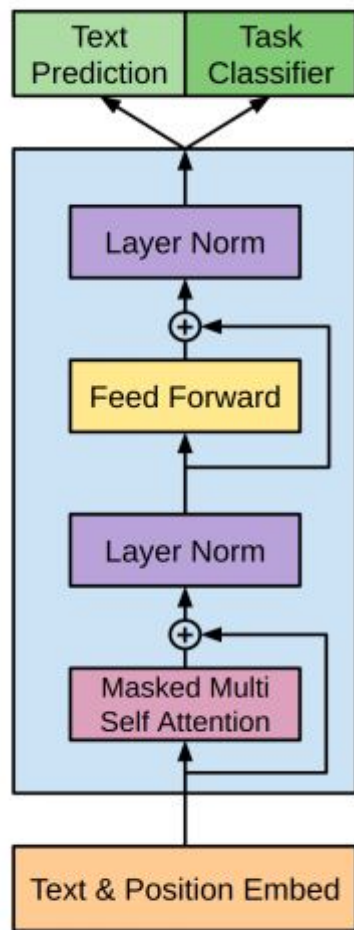
1. Massive amounts of data.
2. Significant computing power.
3. Word embeddings.
4. Self-attention.
5. Transformers.
6. Reinforcement Learning from Human Feedback (RLHF).

GPT-3 Transformer



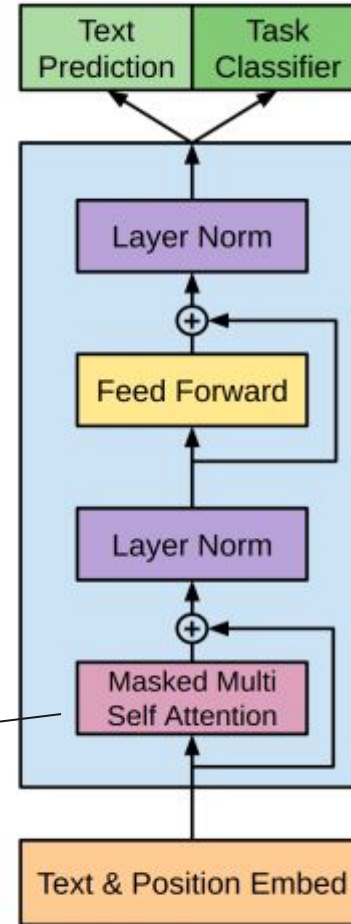
GPT-3 Transformer

Get text as input, compute embedding vectors and transform them based on word positioning in the context.

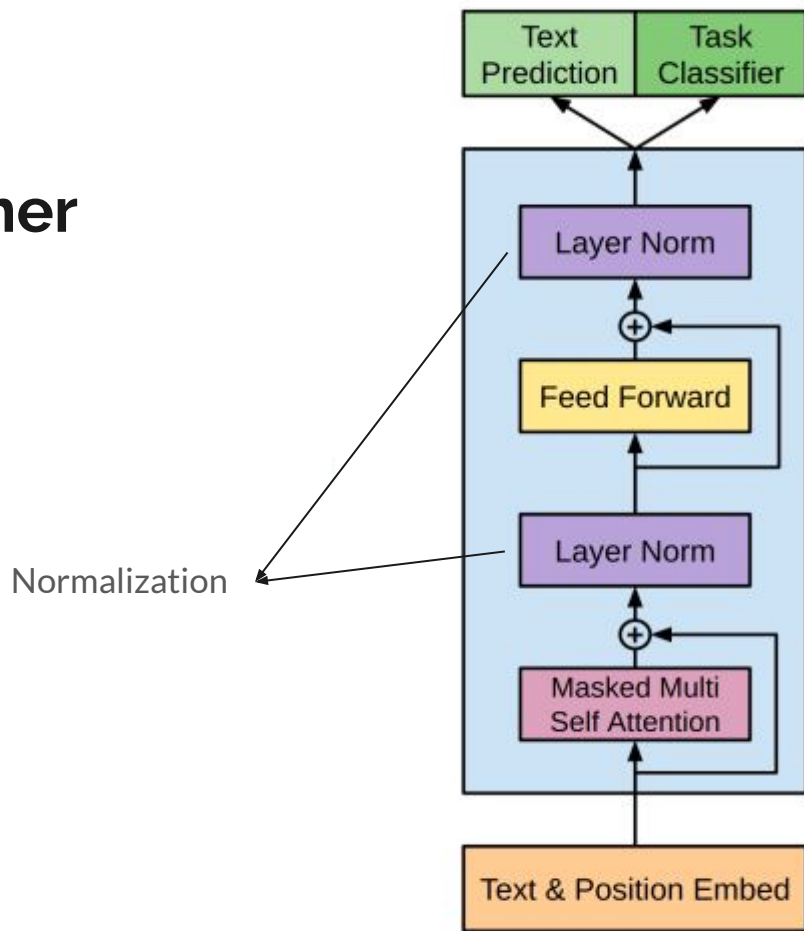


GPT-3 Transformer

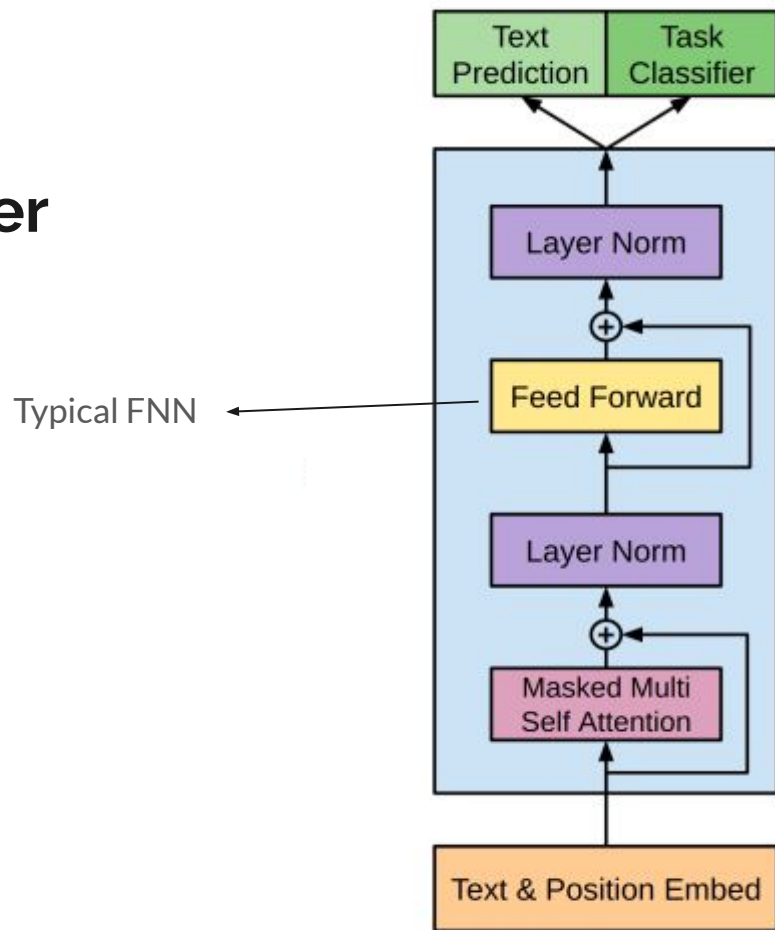
Multiple self-attention heads, each one learning to focus on different types of relationships or patterns in the sequence (e.g., syntax, semantics, long-range dependencies, etc).



GPT-3 Transformer

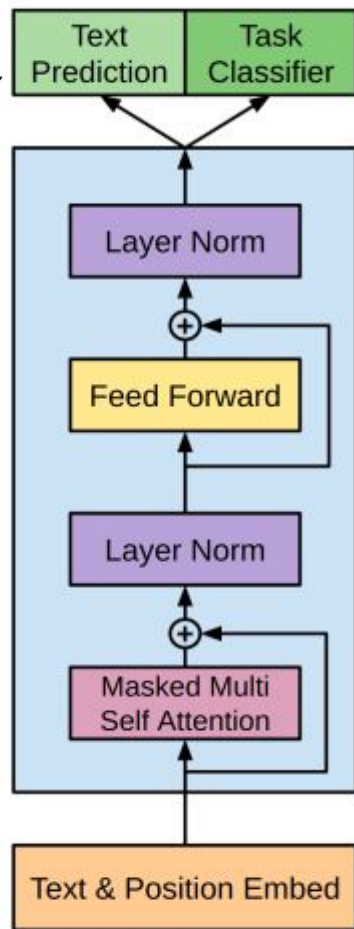


GPT-3 Transformer



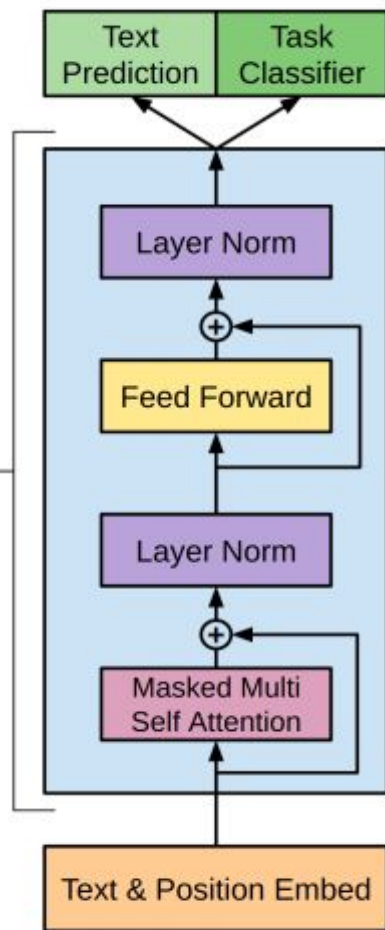
GPT-3 Transformer

Softmax of tokens,
predicting the most
likely next word.



GPT-3 Small Transformer

12 transformer layers.



GPT 3 – Architecture

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}
GPT-3 Small	125M	12	768	12
GPT-3 Medium	350M	24	1024	16
GPT-3 Large	760M	24	1536	16
GPT-3 XL	1.3B	24	2048	24
GPT-3 2.7B	2.7B	32	2560	32
GPT-3 6.7B	6.7B	32	4096	32
GPT-3 13B	13.0B	40	5140	40
GPT-3 175B or “GPT-3”	175.0B	96	12288	96

*Fixed context window of 2,048 for all models.



\$100 dollar bill



\$10,000

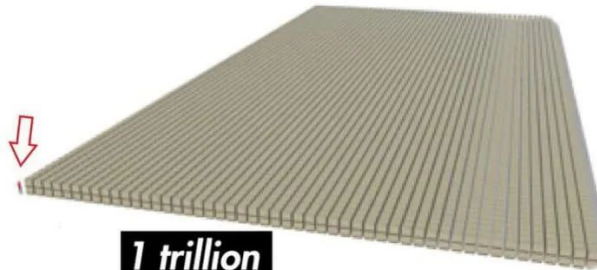
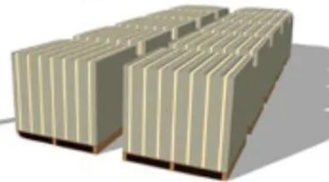


1,000,000



100,000,000

1 Billion



1 trillion



GPT 3 – Full Architecture

- https://dugas.ch/artificial_curiosity/img/GPT_architecture/fullarch.png



GPT – Training

	Pre-training	Post-training
Data quantity	~ the whole internet (trillions of words)	Small (millions?)
Data quality	Low	High
Goal	Most “learning” occurs	Makes model usable



GPT 3 – Tokens

GPT-3 training data^{[1]:9}

Dataset	# tokens	Proportion within training
Common Crawl	410 billion	60%
WebText2	19 billion	22%
Books1	12 billion	8%
Books2	55 billion	8%
Wikipedia	3 billion	3%



Resources

- <https://www.linkedin.com/pulse/gpt-4-explaining-self-attention-mechanism-fatos-ismali/>
- <https://towardsdatascience.com/all-you-need-to-know-about-attention-and-transformers-in-depth-understanding-part-1-552f0b41d021>
- https://docs.google.com/presentation/d/1ZXFIhYczos679r70Yu8vV9uO6B1J0ztzeDxbnBxD1S0/mobilepresent?pli=1&slide=id.g31364026ad_3_2



The success of ChatGPT

1. Massive amounts of data.
2. Significant computing power.
3. Word embeddings.
4. Self-attention.
5. Transformers.
6. Reinforcement Learning from Human Feedback (RLHF).



RLHF

- Trains a model based on human feedback.
- Good human feedback → High reward (low error).
- Bad human feedback → Low reward (high error).



Well-known Architectures using Transformers

- GPT3
- BERT
- T5
- LLaMa
- ...
- <https://huggingface.co/models?other=LLM&sort=trending>



Huggingface

- Use trained models: <https://huggingface.co/learn/nlp-course/chapter1/3>
- Transfer learning: <https://huggingface.co/docs/transformers/training>



SOTA LLM Models

- A lot of optimizations for speeding up training and inference.
- Training refinement, self-correction, etc. for programming and logic.
- Expanded context window.
- Mixture of Experts (MoE).
- Multimodal (text, image, audio, video).